PionCloud

Pion Cloud Model Structure Function Calculator

C++ port of the Hobbs–Melnitchouk Fortran pion-cloud suite

User Manual — Version 1.3

Abstract

PionCloud is a modular C++ library and executable for computing the leading pion-cloud contributions to nucleon structure functions $F_2^{(\pi N)}(x)$ and $F_2^{(\pi N)}(\theta_h)$ in the Sullivan process. It is a direct, validated port of three Fortran programs (`3Var_x.f`, `3Var_theta.f`, `f2_pi_sub.f90`) by T. Hobbs and W. Melnitchouk, extended with configurable scan modes, ROOT output, CTEQ6 neutron structure function evaluation, and a static library interface for embedding in other programs. The code was developed as a framework for BoNuS/TDIS analysis at JLab.

Last updated: April 2026

Contents

1 Introduction **2**
1.1 Physics background . . . 2
1.2 Code history . . . 2

2 Building **2**
2.1 Prerequisites . . . 2
2.2 Full build (ROOT + CTEQ6) . . . 2
2.3 Without ROOT or CTEQ6 . . . 3
2.4 Direct g++ (no CMake) . . . 3
2.5 CTEQ6 table files . . . 3

3 Running **3**
3.1 Quick start . . . 3
3.2 Scan modes . . . 3
3.2.1 `x` scan . . . 4
3.2.2 `theta` scan . . . 4
3.2.3 `kbin` scan . . . 4

4 Input file reference **4**
4.1 Beam kinematics . . . 4
4.2 Physics — channel selection . . . 5
4.3 Physics — form factor . . . 5
4.4 Physics — cutoff parameters . . . 5
4.5 Physics — nucleon and PDF version . . . 5
4.6 Run control and usability . . . 6
4.7 theta-scan settings . . . 6
4.8 kbin-scan settings . . . 6
4.9 Output settings . . . 7

5 Output files **7**
5.1 ROOT objects . . . 7

6 What's new in v1.3 **7**
6.1 Nucleon selector . . . 7
6.2 GRV99 pion PDF . . . 7
6.3 Precision preset . . . 8
6.4 Inclusive mode shortcut . . . 8
6.5 Verbosity control . . . 8
6.6 Angular cut in degrees . . . 8
6.7 CTEQ6 table directory . . . 8
6.8 Single-point eval() for generator use . . . 8
6.9 C interface for Python/Fortran . . . 9
6.10 Fig. 47 left panel (theta scan, full acceptance) . . . 9
6.11 Fig. 47 right panel (theta scan, constrained acceptance) . . . 9
6.12 piN contribution with BoNuS acceptance (black curve) . . . 10
6.13 Four k-bin curves (change k_min/k_max per run) . . . 10
6.14 rho-N and pi-Delta contributions . . . 10

7 Class and API reference **11**
7.1 RunParams (PhysicsParams.hh) . . . 11
7.2 Calculator (Calculator.hh) . . . 11

7.3 ScanResults and data structures (Results.hh) 12
7.4 InputReader (InputReader.hh) 12
7.5 OutputWriter (OutputWriter.hh) 12
7.6 CTEQ6 (CTEQ6.hh) 13
7.7 Physics functions (SplittingFunctions.hh, GRV.hh, Kinematics.hh) 13

8 Using PionCloud as a library **13**

9 CLI reference **14**

10 Project layout **14**

11 Known issues and TODO **15**

Introduction

Physics background

In the Sullivan process, a nucleon N fluctuates into a virtual meson–baryon pair (MB):

$$N \longrightarrow M + B \tag{1}$$

The leading contributions from the pion cloud are:

$$n \to \pi^- + p \quad \text{(charge exchange, } dis = 0) \tag{2}$$

$$p \to \pi^0 + p \quad \text{(neutral exchange, } dis = 1) \tag{3}$$

with subdominant contributions from ρN and $\pi\Delta$ channels.

The tagged structure function, observed when the recoil proton is detected in a specific momentum and angular window, is:

$$F_2^{(\pi N)}(x;\, \Delta k,\, \Delta\theta_h) = \int_{\Delta k,\, \Delta\theta_h} \rho_{\rm PS}(\theta_h, k)\; f_{\pi N}(y, k_T)\; F_2^{\pi}(x/y,\, Q^2)\; dk\, d\theta_h \tag{4}$$

where $\rho_{\rm PS}$ is the phase-space Jacobian, $f_{\pi N}$ is the pion splitting function, and F_2^{π} is the pion structure function evaluated using GRV92 parton distributions.

The ratio $F_2^{(\pi N)}/F_2^n$ is the core BoNuS observable — it gives the fractional pion-cloud contribution to the neutron structure function for a given experimental acceptance.

Code history

The original Fortran programs were written by T. Hobbs (2013–2014) with modifications by W. Melnitchouk. The C++ port (PionCloud) preserves the physics exactly while adding configurability, a library interface, and ROOT output.

Key references:

- Hobbs, Shanahan, Strikman (HSS): form factor parametrisation and Λ values used as defaults.
- Glück, Reya, Vogt (GRV92): pion parton distributions, Z. Phys. **C53** (1992) 651.
- Pumplin et al. (CTEQ6): nucleon PDFs, JHEP **0207** (2002) 012.

Building

Prerequisites

- CMake ≥ 3.14 or g++ ≥ 7
- C++17 or later
- **gfortran** (for CTEQ6 Fortran library; optional but recommended)
- **ROOT** ≥ 6 (optional; required for `.root` output)

Full build (ROOT + CTEQ6)

```
source /path/to/root/bin/thisroot.sh
cmake -B build -DUSE_ROOT=ON -DUSE_CTEQ6=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
# binary: build/PionCloud
# tests:  build/PionCloudTests
```

Without ROOT or CTEQ6

```
cmake -B build -DUSE_ROOT=OFF -DUSE_CTEQ6=OFF
cmake --build build
```

Without CTEQ6: F_2^n and the ratio are set to zero. Without ROOT: only ASCII output is produced.

Direct g++ (no CMake)

```
# No optional dependencies
g++ -std=c++17 -O3 -Iinclude \
    src/Calculator.cc src/InputReader.cc src/OutputWriter.cc src/main.cc
        \
    -o PionCloud

# With ROOT and CTEQ6
gfortran -O2 -c CTEQ6/Cteq6Pdf-2007.f -o Cteq6Pdf.o
g++ -std=c++17 -O3 -Iinclude -DHAVE_ROOT -DHAVE_CTEQ6 \
    src/Calculator.cc src/InputReader.cc src/OutputWriter.cc src/main.cc
        \
    Cteq6Pdf.o $(root-config --cflags --libs) -lgfortran -o PionCloud
```

CTEQ6 table files

The CTEQ6 library reads grid tables from the *current working directory* at runtime. Place the following files in `CTEQ6/` (CMake copies them to the build directory automatically):

Iset	Name	File
1	CTEQ6M (NLO MSbar, default)	`cteq6m.tbl`
2	CTEQ6D (NLO DIS)	`cteq6d.tbl`
3	CTEQ6L (LO)	`cteq6l.tbl`
4	CTEQ6L1 (LO, α_s fitted)	`cteq6l1.tbl`

Running

Quick start

```
# 1. Generate a documented template input file
./PionCloud --write-template myrun.in

# 2. Edit the template, then run
./PionCloud --input myrun.in

# 3. Override any parameter from the command line
./PionCloud --input myrun.in --kmin 0.100 --kmax 0.200 --out Run_100_200
```

Scan modes

PionCloud has four scan modes, selected by `scan_mode`:

Mode	Output	Integrated over	Mirrors
`x`	$F_2^{(\pi N)}(x)$	$(\theta_h, \lvert\vec{k}\rvert)$	`3Var_x.f`
`theta`	$F_2^{(\pi N)}(\theta_h)$	$(x, \lvert\vec{k}\rvert)$	`3Var_theta.f`
`kbin`	$F_2^{(\pi N)}(x; \Delta k, \Delta\theta_h)$	(y, k_T) with $\lvert\vec{k}\rvert$ and $\cos\phi_k$ gates	`f2_pi_sub.f90`
`all`	all of the above	—	—

3.2.1 `x` scan

Integrates over the hadron production angle θ_h and the recoil proton momentum magnitude $|\vec{k}|$ at each output Bjorken-x point. Matches `3Var_x.f` exactly. Useful for computing $F_2^{(\pi N)}(x)$ as a function of x for a given $|\vec{k}|$ tagging window.

3.2.2 `theta` scan

Integrates over x and $|\vec{k}|$ at each output angle θ_h. Matches `3Var_theta.f` exactly. Used to produce Fig. 47-style plots of the angular distribution $F_2^{(\pi N)}(\theta_h; \Delta x, \Delta k)$. The internal x and $|\vec{k}|$ integration ranges are controlled by `x_min_th`, `x_max_th`, `k_min_th`, `k_max_th`:

- **Left panel** (Fig. 47): use defaults ($\Delta x = [0, 0.6]$, $\Delta k = [0, 500]$ MeV).
- **Right panel** (Fig. 47): set `x_min_th=0.05`, `k_min_th=0.060`, `k_max_th=0.250`.

3.2.3 `kbin` scan

Mirrors the `f2_pi_sub.f90` subroutine. Integrates over (y, k_T) at each output x point with:

1. An angular acceptance gate on $\cos\phi_k \in$ [`cosph_min`, `cosph_max`] (default: $\cos 70°$–$\cos 30°$, i.e. [0.342, 0.866]).
2. A $|\vec{k}|$ magnitude window [`k_min`, `k_max`].

The $|\vec{k}|$ magnitude is computed analytically from (k_T, y):

$$|\vec{k}| = \sqrt{k_T^2 + \frac{\left[k_T^2 + (1-(1-y)^2)m_N^2\right]^2}{4\,m_N^2\,(1-y)^2}} \tag{5}$$

This mode is used to produce the four $|\vec{k}|$-bin curves and the individual channel contributions (πN, ρN, $\pi\Delta$).

Input file reference

The input file uses a simple INI-style format. Lines beginning with `#` are comments. Section headers (`[...]`)) are accepted but ignored. All keys are case-insensitive.

Beam kinematics

Key	Default	Description
`beam_energy`	11.0	Electron beam energy E [GeV]
`electron_angle`	35.0	Electron scattering angle θ_e [deg]

Note: `3Var_theta.f` and `3Var_x.f` hardcode $\theta_e = 35°$. The `f2_pi_sub` subroutine (kbin mode) was called with $\theta_e = 12°$ in `f2_pi_ranges.c`. Use 12° for the kbin curves matching the published BoNuS figures.

Physics — channel selection

Key	Default	Description
`contribution`	`pion`	Meson-baryon channel: `pion` (0), `rho` (1), `pi_delta` (2)
`dissociation`	`charge_exchange`	Charge config: `charge_exchange` (0), `neutral` (1)

Charge exchange (*dis*=0): $n \to \pi^- + p$. Isospin factor = 2. Used for the BoNuS $|\vec{k}|$-bin curves.

Neutral (*dis*=1): $p \to \pi^0 + p$. Isospin factor = 1. Used for Fig. 47.

Physics — form factor

Code	Name	Notes
0	`monopole`	$(L^2 + m_N^2)/(L^2 + s_{\pi N})$
1	`dipole`	monopole squared
2	`exp_s`	$\exp[(m_N^2 - s_{\pi N})/L^2]$; HSS default
3	`cov_dipole`	covariant dipole in t
4	`dipole_sch`	dipole, s-channel Λ exchange
5	`exp_t`	$\exp[(t - m_\pi^2)/L^2]$
6	`pauli_villar`	Pauli–Villar; **C++ default**

Physics — cutoff parameters

Key	Default	Description
`lambda`	1.33	Λ for $\pi N/\rho N$ vertex [GeV]
`lambda_delta`	1.39	Λ for $\pi\Delta$ vertex [GeV]

For the HSS exponential form factor (`exp_s`), the recommended values are:

Value	Meaning
$\Lambda = 1.56$ GeV	HSS central value
$\Lambda = 1.63$ GeV	HSS $+1\sigma$
$\Lambda = 1.48$ GeV	HSS -1σ

Physics — nucleon and PDF version

Key	Default	Description
`nucleon`	`neutron`	`neutron` or `proton` for F_2 ratio denominator
`grv_version`	`92`	Pion PDF: `92` (GRV92) or `99` (GRV99)

Run control and usability

Key	Default	Description
`scan_mode`	`all`	`x`, `theta`, `kbin`, or `all`
`precision`	`custom`	`fast`, `normal`, `publication`, `custom`
`inclusive_mode`	`false`	Remove all kinematic cuts (fully inclusive)
`verbose`	`true`	Print per-point progress to stdout
`angle_min_deg`	—	Alias: lower polar angle [deg] → `cosph_max`
`angle_max_deg`	—	Alias: upper polar angle [deg] → `cosph_min`

Key	Default	Description
`x_min`	0.02	Minimum Bjorken-x
`x_max`	0.30	Maximum Bjorken-x
`nx`	100	Number of output x points
`n_theta_integration`	100	Internal θ_h steps

theta-scan settings

Key	Default	Description
`theta_min`	0.0	Lower θ_h bound [deg]
`theta_max`	100.0	Upper θ_h bound [deg]
`nth`	100	Number of output θ_h points
`n_x_integration`	100	Internal x steps
`x_min_th`	10^{-4}	Internal x lower bound
`x_max_th`	0.6	Internal x upper bound
`k_min_th`	10^{-4}	Internal $\lvert\vec{k}\rvert$ lower bound [GeV]
`k_max_th`	0.5	Internal $\lvert\vec{k}\rvert$ upper bound [GeV]

kbin-scan settings

Key	Default	Description
`k_min`	0.050	Lower $\lvert\vec{k}\rvert$ bin bound [GeV]
`k_max`	0.100	Upper $\lvert\vec{k}\rvert$ bin bound [GeV]
`cosph_min`	0.342	Min $\cos\phi_k$ ($= \cos 70°$)
`cosph_max`	0.866	Max $\cos\phi_k$ ($= \cos 30°$)
`ny`	100	Steps in y integration
`nkt`	50000	Steps in k_T integration

Performance note: `nkt=50000` matches the Fortran exactly but is slow. Recommended values:

- `nkt=1000`: quick checks ($\sim$ 5–10% accuracy)
- `nkt=5000`: good accuracy ($<$ 1%)
- `nkt=50000`: publication quality (matches Fortran)

Output settings

Key	Default	Description
`output_mode`	`all`	`ascii`, `root`, `all`, `none`
`output_name`	`PionCloud`	File stem for output files

Output files

With `output_name = MyRun`:

File	Content
`MyRun_params.txt`	Full run parameters + first moments
`MyRun_xscan.dat`	`x`, Q^2, $F_2^{(\pi N)}$, running moment [, F_2^n, ratio]
`MyRun_thetascan.dat`	θ_h, $F_2^{(\pi N)}$, running moment
`MyRun.root`	TGraphs, TH1s, TCanvases, TNamed parameters

ROOT objects

Object names encode the channel and scan mode using the convention `<channel>_<mode>`:

- channel: `piN`, `piN_neutral`, `rhoN`, `piDelta`
- mode: `xscan`, `theta`, `kbin`

Name	Type	Description
`g_<pfx>`	TGraph	$F_2^{(\text{channel})}(x)$
`g_moment_<pfx>`	TGraph	Running $\int F_2\,dx$
`h_<pfx>`	TH1D	Same, as histogram
`c_<pfx>`	TCanvas	Log-scale plot
`g_F2neutron`	TGraph	$F_2^n(x)$ from CTEQ6M (always this name)
`g_ratio_<pfx>`	TGraph	$F_2^{(\text{channel})}/F_2^n$
`h_ratio_<pfx>`	TH1D	Ratio as histogram
`c_ratio_<pfx>`	TCanvas	Two-pad: curves (top) + ratio (bottom)
`g_thetascan`	TGraph	$F_2^{(\pi N)}(\theta_h)$
`g_moment_theta`	TGraph	Running $\int F_2\,d\theta$
`h_thetascan`	TH1D	Same, as histogram
`c_thetascan`	TCanvas	Angular distribution plot
`RunParams`	TNamed	Full parameter summary string

What's new in v1.3

Nucleon selector

The reference nucleon for the ratio $F_2^{(\pi N)}/F_2^{\text{nucleon}}$ can now be chosen:

```
nucleon = neutron     # default: F2n via charge symmetry (matches Fortran
    )
nucleon = proton      # F2p directly from CTEQ6
```

GRV99 pion PDF

An updated 1999 pion PDF parametrisation is available alongside the default GRV92:

```
grv_version = 92     # default: GRV92, matches original Fortran
grv_version = 99     # GRV99 update from f2_pi_sub.f90
```

Precision preset

Sets `ny` and `nkt` together for the kbin scan:

```
precision = fast           # ny=20,  nkT=500   -- quick checks
precision = normal         # ny=50,  nkT=2000  -- development
precision = publication    # ny=100, nkT=50000 -- matches Fortran (
    default)
precision = custom         # use ny/nkT directly
```

Inclusive mode shortcut

Removes all kinematic cuts (reproduces the fully inclusive $F_2^{(\pi N)}(x)$ purple curve without manually setting four parameters):

```
inclusive_mode = true
```

Equivalent to setting `cosph_min=0`, `cosph_max=1`, `k_min=0.001`, `k_max=10.0`.

Verbosity control

```
verbose = false     # suppress all stdout (for library/generator
    embedding)
```

Angular cut in degrees

Aliases for `cosph_min`/`cosph_max`:

```
angle_min_deg = 30.0     # lower polar angle -> cosph_max = cos(30) =
    0.866
angle_max_deg = 70.0     # upper polar angle -> cosph_min = cos(70) =
    0.342
```

CTEQ6 table directory

Allows running from an arbitrary working directory:

```
p.cteq6_table_dir = "/path/to/cteq6_tables";
// Calculator chdir()s there before CTEQ6::init(), then restores cwd.
```

Single-point eval() for generator use

```
Calculator calc(p);
double f2pi = calc.eval(x, Q2);    // Q2 <= 0: computed from E,x,theta_e
```

Returns $F_2^{(\pi N)}(x, Q^2)$ for one point. Always silent regardless of `p.verbose`. Suitable for event-by-event generator calls.

C interface for Python/Fortran

```c
#include "pioncloud_c.h"
pioncloud_init_default();                    // default BoNuS config
double f2pi = pioncloud_eval_f2pi(x, Q2);
double f2n  = pioncloud_eval_f2n (x, Q2);
double rat  = pioncloud_eval_ratio(x, Q2);
```

Python usage via `ctypes`:

```python
import ctypes
lib = ctypes.CDLL("./libPionCloudLib.so")
lib.pioncloud_init_default.restype = None
lib.pioncloud_eval_f2pi.restype    = ctypes.c_double
lib.pioncloud_eval_f2pi.argtypes   = [ctypes.c_double, ctypes.c_double]
lib.pioncloud_init_default()
f2pi = lib.pioncloud_eval_f2pi(0.1, 2.0)
```

See `include/pioncloud_c.h` for full Fortran interface documentation.

Thread safety warning: CTEQ6 uses Fortran `COMMON` blocks and is not thread-safe. Do not call from multiple threads simultaneously.

Fig. 47 left panel (theta scan, full acceptance)

```
scan_mode   = theta
output_name = Fig47_left
contribution = pion
dissociation = neutral      # p -> pi0 + p
form_factor  = exp_s
lambda       = 1.56
beam_energy    = 11.0
electron_angle = 35.0       # as in 3Var_theta.f
theta_min = 0.0
theta_max = 90.0
nth       = 100
n_x_integration = 100
# defaults: x in [0, 0.6], k in [0, 500] MeV -- left panel
```

Fig. 47 right panel (theta scan, constrained acceptance)

```
scan_mode   = theta
output_name = Fig47_right
contribution = pion
dissociation = neutral
form_factor  = exp_s
lambda       = 1.56
beam_energy    = 11.0
electron_angle = 35.0
theta_min = 0.0
theta_max = 90.0
nth       = 100
n_x_integration = 100
x_min_th  = 0.05            # Delta_x = [0.05, 0.6]
x_max_th  = 0.6
k_min_th  = 0.060           # Delta_k = [60, 250] MeV
k_max_th  = 0.250
```

piN contribution with BoNuS acceptance (black curve)

```
scan_mode      = kbin
output_name    = piN_channel
contribution   = pion
dissociation   = charge_exchange
form_factor    = exp_s
lambda         = 1.56
beam_energy    = 11.0
electron_angle = 12.0        # as used in f2_pi_ranges.c
x_min = 0.055
x_max = 0.30
nx    = 20
k_min     = 0.060            # BoNuS full acceptance window
k_max     = 0.250
cosph_min = 0.342            # 30-70 degree cut
cosph_max = 0.866
ny    = 100
nkt   = 5000
```

Four k-bin curves (change k_min/k_max per run)

```
# Run four times changing k_min and k_max:
# Run 1: k_min=0.060, k_max=0.100  (red)
# Run 2: k_min=0.100, k_max=0.200  (green)
# Run 3: k_min=0.200, k_max=0.300  (blue)
# Run 4: k_min=0.300, k_max=0.400  (yellow)
scan_mode      = kbin
contribution   = pion
dissociation   = charge_exchange
form_factor    = exp_s
lambda         = 1.56
beam_energy    = 11.0
electron_angle = 12.0
x_min = 0.055
x_max = 0.33
nx    = 20
k_min = 0.060   # change per run
k_max = 0.100   # change per run
cosph_min = 0.342
cosph_max = 0.866
ny  = 100
nkt = 5000
```

rho-N and pi-Delta contributions

To reproduce the red (ρN) and green ($\pi\Delta$) curves, use the same configuration as the piN example above but change:

```
# rho-N (red dashed):
contribution = rho

# pi-Delta (green dash-dot):
contribution = pi_delta
lambda       = 1.39     # use lambda_delta for pi-Delta
```

Class and API reference

RunParams (PhysicsParams.hh)

The central configuration struct. All fields have sensible defaults. Can be constructed and filled entirely in C++ without an input file.

```
#include "PhysicsParams.hh"
PionCloud::RunParams p;
p.flag        = PionCloud::FLAG_PION;
p.typ         = PionCloud::FF_EXP_S;
p.L           = 1.56;
p.dis         = PionCloud::DIS_CHARGE_EXCHANGE;
p.E           = 11.0;
p.theta_e     = 12.0;
p.kmin        = 0.060;
p.kmax        = 0.250;
p.cosph_min   = 0.342;
p.cosph_max   = 0.866;
p.ny          = 100;
p.nkT         = 5000;
p.outputMode  = PionCloud::OUT_NONE;  // no file I/O
```

Key enumerations:

Enum	Values	Key
`FormFactorType`	`FF_MONOPOLE..FF_PAULI_VILLAR`	`form_factor`
`ContributionFlag`	`FLAG_PION, FLAG_RHO, FLAG_PI_DELTA`	`contribution`
`DisChannel`	`DIS_CHARGE_EXCHANGE, DIS_NEUTRAL`	`dissociation`
`OutputMode`	`OUT_NONE, OUT_ASCII, OUT_ROOT, OUT_ALL`	`output_mode`

Calculator (Calculator.hh)

The integration engine. No I/O dependencies.

```
#include "Calculator.hh"
PionCloud::Calculator calc(p);

PionCloud::ScanResults res;
calc.runXScan(res);          // x-scan (3Var_x.f)
calc.runThetaScan(res);      // theta-scan (3Var_theta.f)
calc.runKBinScan(res);       // kbin-scan (f2pi_sub.f90)

// Or run the mode selected by p.scanMode:
PionCloud::ScanResults res2 = calc.runAll();
```

Public methods:

Method	Description
`runXScan(out)`	x-scan; fills `out.xScan`
`runThetaScan(out)`	theta-scan; fills `out.thetaScan`
`runKBinScan(out)`	kbin-scan; fills `out.xScan`
`runAll()`	runs modes selected by `p_.scanMode`; returns `ScanResults`

Private static helpers:

Method	Description
`kMag(kT, y)`	3D recoil proton momentum magnitude from (k_T, y)
`cosPhi(kT, y)`	Proton polar angle $\cos\phi_k$ from (k_T, y)

ScanResults and data structures (Results.hh)

Plain structs with no ROOT or file-system dependencies:

```
struct XScanPoint {
    double x;            // Bjorken-x
    double Q2;           // Q^2 at this x [GeV^2]
    double F2piK;        // F2^pi(x) integrated over phase space
    double F2piMoment;   // running integral
    double F2n;          // F2^n from CTEQ6 (0 if unavailable)
    double ratio;        // F2piK / F2n
};

struct ThetaScanPoint {
    double theta_deg;    // hadron production angle [degrees]
    double F2piK;        // F2^pi(theta_h)
    double F2piMoment;   // running integral
};

struct ScanResults {
    std::vector<XScanPoint>     xScan;
    std::vector<ThetaScanPoint> thetaScan;
    double F2pi0_x     = 0.0;  // first moment (Simpson)
    double F2pi0_theta = 0.0;
};
```

InputReader (InputReader.hh)

Parses INI-style input files. Optional — `RunParams` can be set directly in C++ without using this class.

```
#include "InputReader.hh"
PionCloud::InputReader reader;
PionCloud::RunParams p = reader.read("myrun.in");

if (reader.hasErrors()) {
    reader.printDiagnostics();
    return 1;
}

// Generate a fully documented template file:
PionCloud::InputReader::writeTemplate("myrun.in");
```

OutputWriter (OutputWriter.hh)

Writes ASCII and/or ROOT output. Controlled by `p.outputMode`.

```
#include "OutputWriter.hh"
PionCloud::OutputWriter writer(p, "MyRun");
writer.write(results);
```

CTEQ6 (CTEQ6.hh)

C++ wrapper for the CTEQ6 Fortran PDF library. Handles all float/double casts internally.

```
#include "CTEQ6.hh"
PionCloud::CTEQ6::init();                          // call once
double f2n = PionCloud::CTEQ6::F2neutron(x, Q2);
double f2p = PionCloud::CTEQ6::F2proton (x, Q2);
double xfu = PionCloud::CTEQ6::xfx(1, x, Q);    // u-quark
```

Available functions:

Function	Description
`init(set)`	Initialise PDF tables (default: CTEQ6M, iset=1)
`xfx(iparton,x,Q)`	$x\,f(x,Q)$ for parton code `iparton`
`F2neutron(x,Q2)`	F_2^n via charge symmetry
`F2proton(x,Q2)`	F_2^p directly from CTEQ6

Parton codes: 0=gluon, 1=u, 2=d, 3=s, 4=c, 5=b; negative values for antiquarks.

Physics functions (SplittingFunctions.hh, GRV.hh, Kinematics.hh)

All inline, header-only, no dependencies:

Function	Description
`fypiN(y,kT,L,typ,dis)`	πN splitting function (TOPT)
`f_rhoN(y,kT,L,typ,dis)`	ρN splitting function
`f_RhoDel(y,kT,L,typ,dis)`	$\rho\Delta$ splitting function
`fypiD(y,kT,Ld,typ,dis)`	$\pi\Delta$ splitting function (SU(4))
`GRV92(xi,Q2,xV,xS)`	GRV92 LO pion PDFs
`F2piGRV(xi,Q2)`	$(5/9)(xV + 2xS)$: electromagnetic combination
`calcQ2(E,x,theta_e)`	Q^2 from beam kinematics
`simpsonWeight(i,n)`	Composite Simpson weight for index i of n

Using PionCloud as a library

PionCloud is built as a static library `libPionCloudLib.a`. Any C++ program can embed it:

```
# In your CMakeLists.txt:
add_subdirectory(PionCloud)
target_link_libraries(MyProgram PRIVATE PionCloudLib)
```

```
#include "PhysicsParams.hh"
#include "Calculator.hh"
#include "Results.hh"

using namespace PionCloud;

RunParams p;
p.flag       = FLAG_PION;
p.typ        = FF_EXP_S;
p.L          = 1.56;
p.dis        = DIS_CHARGE_EXCHANGE;
p.E          = 11.0;  p.theta_e = 12.0;
p.kmin       = 0.060; p.kmax    = 0.250;
```

```
p.cosph_min  = 0.342; p.cosph_max = 0.866;
p.ny  = 100;  p.nkT = 5000;
p.outputMode = OUT_NONE;   // no files written
p.scanMode   = "kbin";

Calculator calc(p);
ScanResults res;
calc.runKBinScan(res);

for (auto &pt : res.xScan)
    printf("x=%.4f  Q2=%.4f  F2piK=%.6e  F2n=%.6e  ratio=%.6e\n",
           pt.x, pt.Q2, pt.F2piK, pt.F2n, pt.ratio);
```

Current limitations for generator use (see TODO.md):

- No single-point `eval(x, Q2)` method: full scan must be run.
- No verbosity control: progress is printed to `stdout`.
- CTEQ6 reads table files from the current working directory.
- CTEQ6 is not thread-safe (Fortran `COMMON` blocks).

CLI reference

```
./PionCloud [--input FILE] [OPTIONS]

--input FILE              Read parameters from INI file
--write-template FILE     Write documented template and exit

--mode x|theta|kbin|all   Scan mode
--output ascii|root|all|none  Output format
--out STEM                Output file stem

# Physics
--flag 0|1|2              Contribution (pion/rho/pi_delta)
--dis  0|1                Dissociation channel
--typ  0..6               Form factor type
--L    GeV                Lambda (pi/rho-N)
--Ld   GeV                Lambda (Delta)
--E    GeV                Beam energy
--theta_e DEG             Electron scattering angle

# x-scan
--xmin --xmax --nx --nth-int

# theta-scan
--thmin --thmax --nth --nx-int
--xmin-th --xmax-th --kmin-th --kmax-th

# kbin-scan / |k| window
--kmin --kmax --nk
--ny --nkt --cosph-min --cosph-max
```

Project layout

```
PionCloud/
|---- CMakeLists.txt
```

```
|---- README.md
|---- TODO.md
|---- Fig47_left.in          # ready-to-use input: Fig.47 left panel
|---- Fig47_right.in         # ready-to-use input: Fig.47 right panel
|---- CTEQ6/
|     |---- Cteq6Pdf-2007.f    # Fortran PDF library (v6.52)
|     |---- cteq6m.tbl         # iset=1 NLO MSbar (default)
|     |---- cteq6d.tbl / cteq6l.tbl / cteq6l1.tbl
|     `---- README
|---- include/
|     |---- PhysicsParams.hh   # RunParams, enums
|     |---- SplittingFunctions.hh  # fypiN, f_rhoN, f_RhoDel, fypiD
|     |---- GRV.hh             # GRV92 pion PDFs
|     |---- Kinematics.hh      # Q2, phase space, Simpson weights
|     |---- Results.hh         # XScanPoint, ThetaScanPoint, ScanResults
|     |---- Calculator.hh      # main integration engine
|     |---- CTEQ6.hh           # extern C bridge to Fortran
|     |---- InputReader.hh     # INI file parser
|     `---- OutputWriter.hh    # ASCII + ROOT writer
|---- src/
|     |---- Calculator.cc
|     |---- InputReader.cc
|     |---- OutputWriter.cc
|     `---- main.cc
|---- tests/
|     |---- CMakeLists.txt
|     `---- test_all.cc        # 150 tests, 0 failures
`---- docs/
    `---- PionCloud_manual.tex   # this document
```

Known issues and TODO

The complete list is maintained in `TODO.md`. Summary of high-priority items:

1. **Nucleon selector**: `nucleon = neutron|proton` to choose between F_2^n and F_2^p in the ratio. `F2proton()` already exists in `CTEQ6.hh`; only wiring is needed.
2. **GRV99 pion PDF**: add the 1999 parametrisation as `grv_version = 92|99`.
3. **Theta-scan ROOT names**: `g_thetascan` etc. should use the `<channel>_<mode>` prefix.
4. **kbin `nkt` default is slow**: 50000 steps is publication-quality but takes O(hours) for `nx=100`. Use `nkt=5000` for development.
5. **Generator interface**: single-point `eval()`, verbosity control, configurable CTEQ6 table path, and a C `extern "C"` wrapper for Python/Fortran interoperability.
6. **t-scan mode**: $\mathrm{F}_2^{(\pi N)}$ vs Mandelstam $-t$, binning the (y, k_T) integrand by $t = -k_T^2 - m_N^2 y^2/(1-y)$.